5.1 First Twig render

HTML repeated across pages is difficult to maintain. Twig lets us write the view in a template and provide its changing values from PHP. Start with one greeting so we can see every piece before introducing shared layouts.

Create the template

Create `templates/tutorials/class5/hello.html.twig`:

```
<!doctype html>
<html lang="en">
<head>
    <meta charset="utf-8">
    <title>Campus Club greeting</title>
</head>
<body>
    <h1>Hello {{ name }}</h1>
    <p>Welcome to the Campus Club.</p>
</body>
</html>
```

Double braces output an expression. Twig variable names have no dollar sign. The `.html.twig` extension identifies a Twig template intended to produce HTML; the browser receives the rendered HTML, not this source file.

Create the Application class

Create `src/Tutorials/Class5/Application.php`:

```
<?php
declare(strict_types=1);

namespace App\Tutorials\Class5;

use Twig\Environment;
use Twig\Loader\FilesystemLoader;

class Application
{
    private const PATH_TO_TEMPLATES = __DIR__ .
'/../../../templates/tutorials/class5';

    private Environment $twig;

    public function __construct()
    {
```

```
        $loader = new FilesystemLoader(self::PATH_TO_TEMPLATES);
        $this->twig = new Environment($loader, [
            'autoescape' => 'html',
            'strict_variables' => true,
        ]);
    }

    public function run(): void
    {
        $template = 'hello.html.twig';
        $args = ['name' => 'Avery'];
        $html = $this->twig->render($template, $args);
        echo $html;
    }
}
```

The loader locates template files. The Environment processes templates and holds our options. `render()` returns a string; `echo` sends it to the response. The key `name` in the context array matches the template variable. The template name is relative to the loader's directory.

Trace the three `..` segments from `src/Tutorials/Class5` to the repository root. `self::` refers to a class constant; `$this->twig` refers to this object's property. Composer loads PHP classes, while FilesystemLoader locates Twig templates. They have different responsibilities.

We explicitly enable HTML autoescaping and strict variables for these lessons. Strict variables reveal misspelled names instead of silently producing empty output. Compiled-template caching is left at its default disabled setting during development.

Add the entry script

Create `public/tutorials/class5/index.php`:

```
<?php
declare(strict_types=1);

use App\Tutorials\Class5\Application;

require_once __DIR__ . '/../../../vendor/autoload.php';

$app = new Application();
$app->run();
```

Use the server and URL from the overview. Expect **Hello Avery**. View page source: it contains the name and ordinary HTML, with no Twig expressions. Change the PHP value to `Morgan & Riley`; expect a readable ampersand in the browser and `&` in the HTML source.

Temporarily set the name to `<strong>Avery</strong>`. Expect literal angle-bracket text, not bold text. Restore Avery. Autoescaping encodes this value for HTML output; it does not validate all possible URLs, JavaScript, or other contexts. Do not add `raw` to make this demonstration render as markup.

Reference: Twig developer API and options.